

一、`io_bound` 函数的核心定位

`nicegui.run` 模块下的 `io_bound` 函数是 NiceGUI 专为 **I/O 密集型任务** 设计的异步执行工具，其核心目标是将耗时的 I/O 操作（如网络请求、文件读写、数据库查询等）卸载到独立线程中执行，避免阻塞 NiceGUI 的主事件循环，从而保证 UI 界面始终响应（不卡顿、不卡死）。

二、核心特性与工作原理

1. 线程模型

- NiceGUI 的主程序运行在异步事件循环（基于 Python `asyncio`）中，而 I/O 密集型任务本身多为同步操作（如 `httpx.get` 同步调用、`open()` 读写文件），若直接在事件循环中执行会阻塞整个 UI。
- `io_bound` 会将传入的同步函数封装后，提交到 NiceGUI 管理的**线程池**中执行，线程池的大小默认适配系统资源，无需手动配置。
- 函数返回一个 `asyncio.Future` 对象，可通过 `await` 关键字异步等待任务完成，获取执行结果（或捕获异常），全程不阻塞事件循环。

2. 入参规则

`io_bound` 的调用格式为：

```
await run.io_bound(func, *args, **kwargs)
```

- `func`：必填，要执行的**同步 I/O 函数**（如 `httpx.get`、`open`、`requests.get`、数据库查询函数等）；
- `*args`：可选，传递给 `func` 的位置参数（如示例中的 `URL`）；
- `**kwargs`：可选，传递给 `func` 的关键字参数（如示例中的 `timeout=3`）。

3. 返回值

返回 `asyncio.Future` 对象，`await` 该对象后会得到 `func` 函数的执行结果（如示例中 `httpx.get` 返回的响应对象）；若 `func` 执行出错，`await` 时会抛出对应的异常（可通过 `try/except` 捕获）。

三、适用场景

`io_bound` 仅适用于 **I/O 密集型任务**，典型场景包括：

- 网络请求：如 HTTP/HTTPS 请求（`httpx/requests`）、API 调用、爬虫数据获取；
- 文件操作：如大文件的读写、本地文件的复制 / 移动；
- 数据库交互：如 SQL 查询、NoSQL 数据读写（同步数据库驱动）；
- 外部设备通信：如串口通信、网络设备连接。

⚠ 注意：`io_bound` 不适合 CPU 密集型任务（如大规模计算、数据处理），CPU 密集型任务应使用 `run.cpu_bound`（基于进程池），否则线程的 GIL 锁会导致性能瓶颈。

四、使用示例拆解（结合提供的代码）

```
import httpx
from nicegui import run, ui

async def handle_click():
    # 定义要请求的 URL（I/O 任务）
    URL = 'https://httpbin.org/delay/1'
    # 关键：通过 io_bound 执行同步的 httpx.get，传入 URL 和超时参数
    # await 等待线程中执行完成，获取响应对象
    response = await run.io_bound(httpx.get, URL, timeout=3)
    # UI 反馈：显示响应内容的字节数，此时 UI 仍可交互
    ui.notify(f'Downloaded {len(response.content)} bytes')

# 绑定按钮点击事件（异步处理函数）
ui.button('Download', on_click=handle_click)

# 启动 NiceGUI 应用
ui.run()
```

执行流程：

1. 点击“Download”按钮，触发异步函数 `handle_click`；
2. `run.io_bound(httpx.get, URL, timeout=3)` 将 `httpx.get` 提交到线程池，主事件循环继续运行（UI 不卡）；
3. 线程中执行 HTTP 请求（耗时约 1 秒），完成后返回响应对象；
4. `await` 拿到结果，执行 `ui.notify` 更新 UI。

五、异常处理

若 I/O 任务执行失败（如网络超时、文件不存在），需通过 `try/except` 捕获异常，避免 UI 崩溃：

```
async def handle_click():
    URL = 'https://httpbin.org/delay/1'
    try:
        response = await run.io_bound(httpx.get, URL, timeout=3)
        ui.notify(f'Downloaded {len(response.content)} bytes')
    except httpx.TimeoutException:
        ui.notify('Request timed out!', type='error')
    except Exception as e:
        ui.notify(f'Error: {str(e)}', type='error')
```

六、与纯 `asyncio` 线程执行的对比

手动使用 `asyncio.to_thread` 也能实现类似效果，但 `run.io_bound` 有两个优势：

1. 内置线程池管理：无需手动创建 / 销毁线程池，NiceGUI 统一管理，避免资源泄漏；
2. 与 NiceGUI 生命周期适配：线程任务会随应用关闭自动终止，避免残留线程；
3. 简化调用：无需手动封装函数，直接传递目标函数和参数即可。

